Trabajo Colaborativo con GIT



Repaso de Sistemas de Control de Versiones

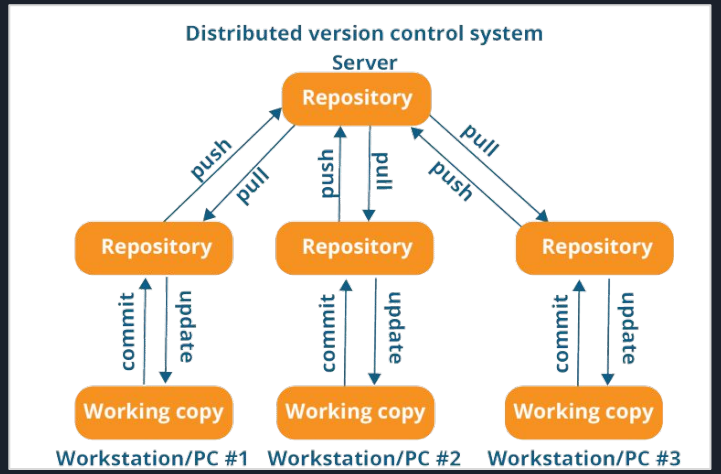
¿Qué es un sistema de control de versiones distribuido (DVCS)?

En este tipo de sistemas de versionado distribuido permiten que los usuarios puedan tener sus propias copias del repositorio locales.

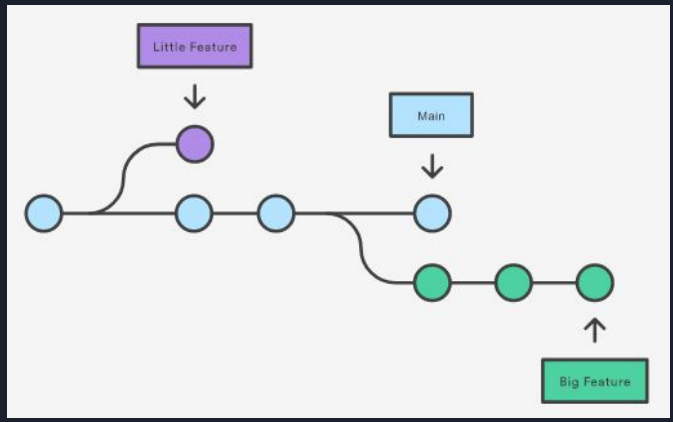
Esto permite:

- Operar sobre el sistema de versionado de manera mucho más rápida siendo que los cambios son locales.
- Ganar mucha más flexibilidad siendo que los cambios en los archivos realizados de manera local no afectan al servidor central.
- Manejar código versionado sin depender de la disponibilidad del servidor.

Git es un DVCS.



Trabajando con Branches

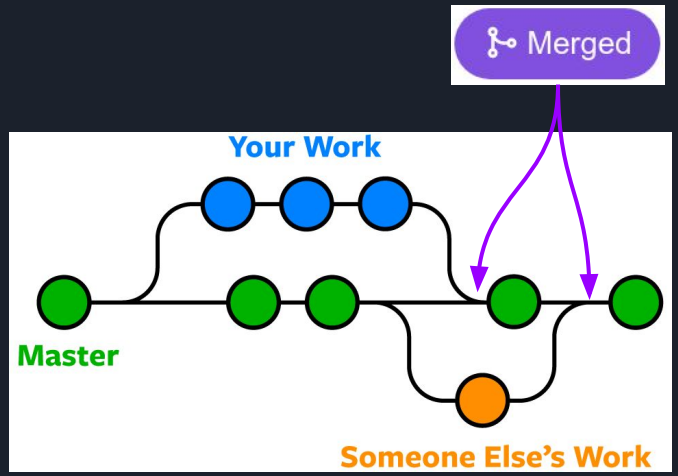


Las branches (o ramas) de Git son referencias a una snapshot en la historia del codebase.
Las mismas presentes en el repositorio local y pueden ser pusheadas al repositorio remoto.

Las branches no tienen costo y permiten trabajar de manera simultánea concurrente en distintos flujos de trabajo.

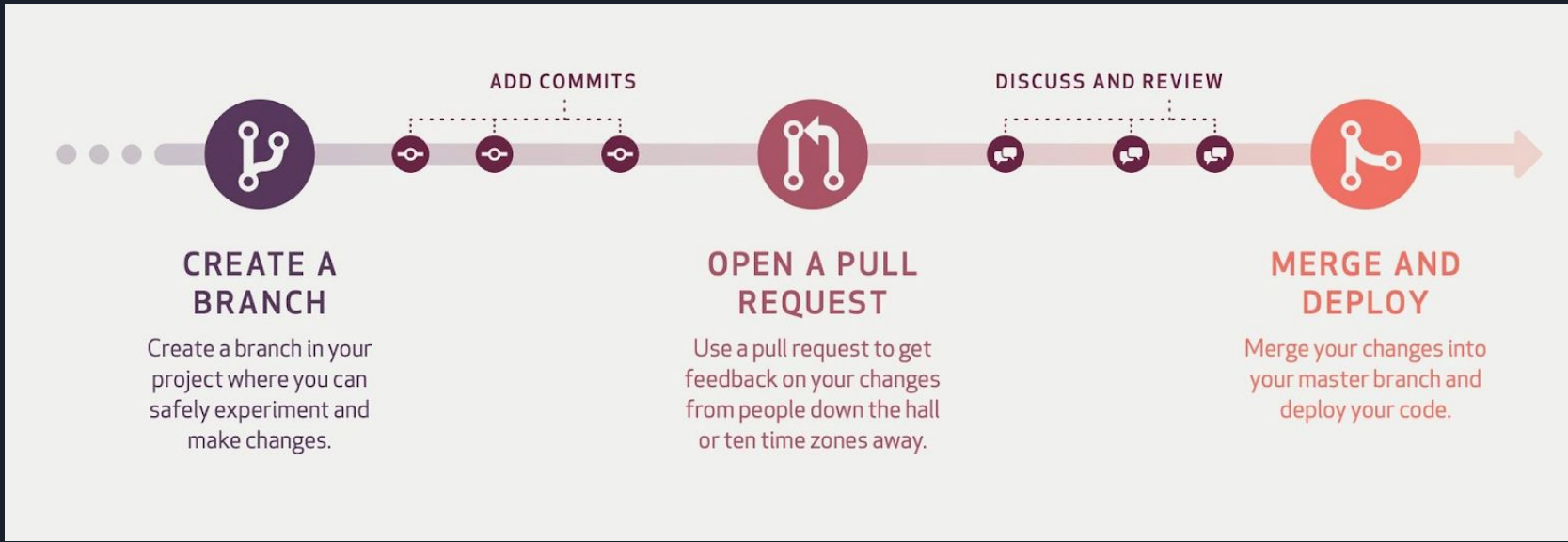
El uso de branches permite ordenar el desarrollo y trabajar en equipo sin temor a afectar el trabajo de otros.

También es posible combinar branches para unificar el trabajo realizado por distintas personas. Esto se llama **mergear** branches.



Pull Requests

¿Como es el workflow para agregar features?



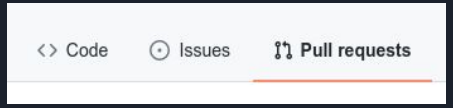
¿Qué es un Pull Request (PR)?

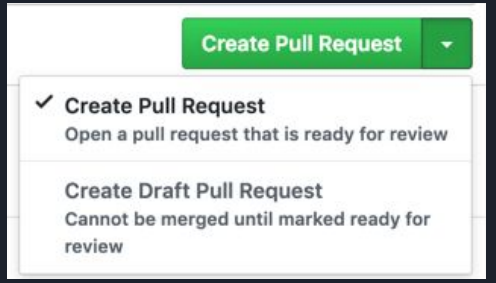
Un PR (aka Merge Request/MR) es una solicitud para cambios de un branch a otro.

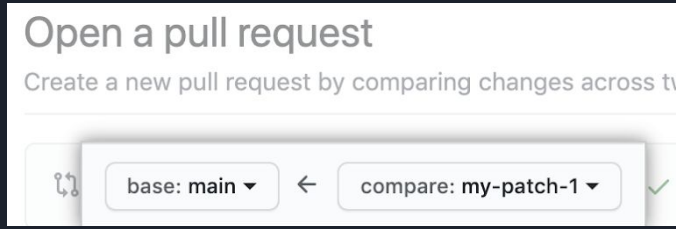
Cuando se crea el PR se debe seleccionar la branch origen de los cambios y la branch destino que va a recibirlos.

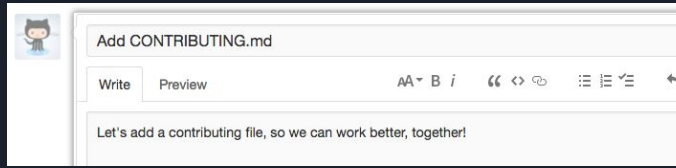
El PR debe contener una descripción de los cambios incluidos.

Es una buena práctica crear el PR como draft apenas uno empieza a trabajar en un feature para que el resto tenga visibilidad.



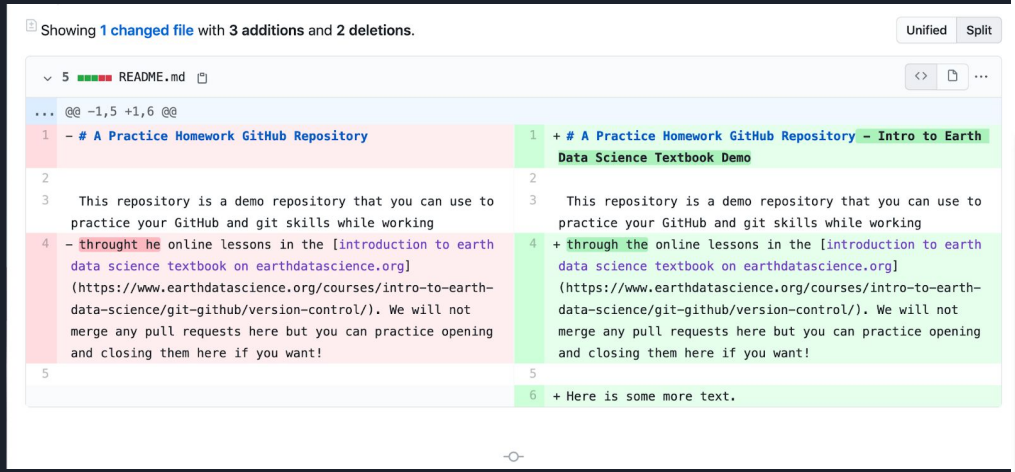


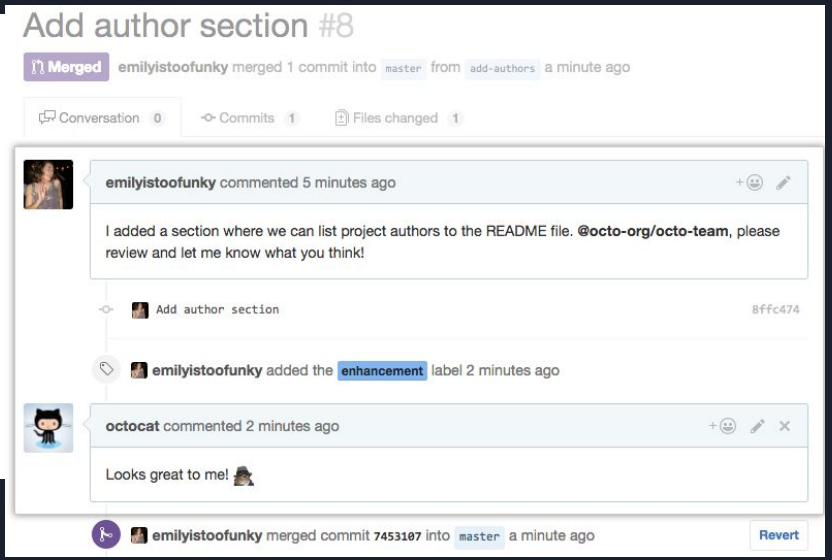




PR reviews

Al crear un PR se abre un espacio en el cual los colaboradores del proyecto puede revisar los cambios introducidos, comentar al respecto y aprobarlos o rechazarlos.





Practica de PRs

WORKSHOP: Desde línea de comandos

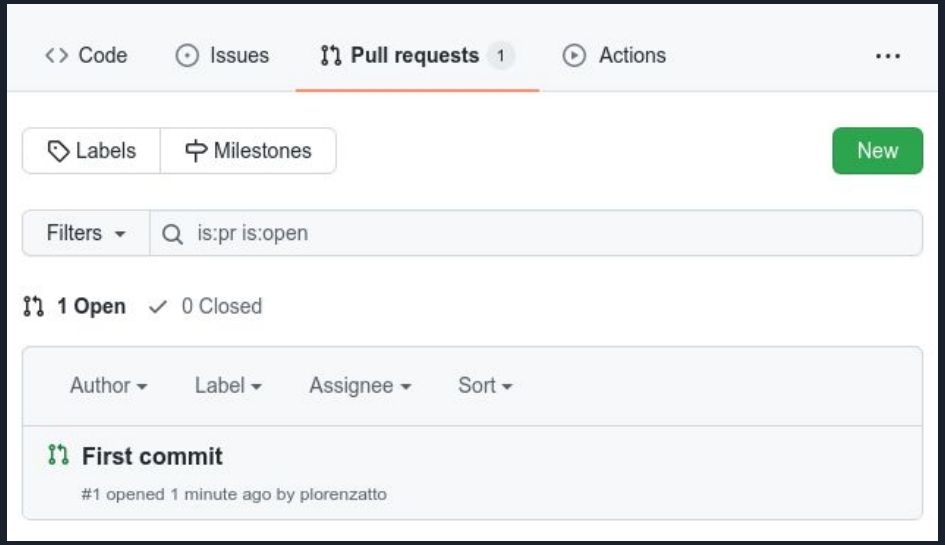
```
$ git checkout -b add-main           # Creacion nuevo branch.
Switched to a new branch 'add-main'

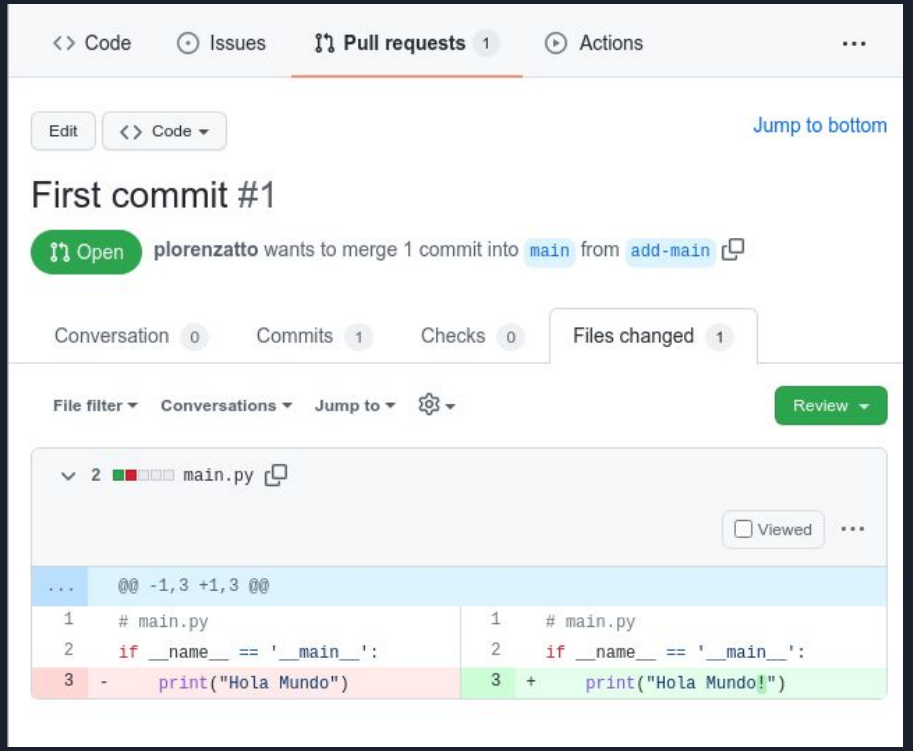
$ cat > main.py <<EOF                # Creacion del archivo.
# main.py
if __name__ == '__main__':
    print("Hola Mundo")
EOF

$ git add main.py                    # Añadir archivo (stage)
$ git commit -m "First commit"       # Comiteamos
[add-main (root-commit) 554fba2] First commit
 1 file changed, 3 insertions(+)
 create mode 100644 main.py

$ git push                           # Pusheamos el cambio
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 276 bytes | 276.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To github.com:plorenzatto/deleteme.git
 * [new branch]      add-main -> add-main
```

WORKSHOP: Create, Clone, Commit, PR, Merge





Manejo de Conflictos

¿Qué es un conflicto?

Al tratar de **mergear** cambios de una branch a otra puede ocurrir que en ambas se haya modificado el mismos archivo en las mismas líneas. Esto genera un **conflicto** entre las dos versiones del mismo archivo.

¿Cómo sabemos si hay un conflicto? (CLI)

En un flujo habitual cuando quieren traer los cambios de un repositorio remoto a su local usando el comando `git pull` puede que se encuentren con algo como lo siguiente:

```
$ git pull origin new_branch_to_merge_later
Auto-merging merge.txt
CONFLICT (content): Merge conflict in merge.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Si corremos `git status` podemos ver algunos archivos como “both modified”:

```
$ git status
On branch main
You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)

Unmerged paths:
(use "git add <file>..." to mark resolution)

both modified:   merge.txt
```

¿Cómo se resuelve un conflicto?

Cuando ocurre un conflicto los archivos conflictivos van a tener marcas indicando las líneas con problemas:

```
$ cat merge.txt
<<<<<<< HEAD
this is some content to mess with
content to append
=======
totally different content to merge later
>>>>>>> new_branch_to_merge_later
```

Tomando esta información podemos editar los archivos afectados para crear un nuevo commit que resuelva los conflictos como si se tratara de cualquier otro cambio.

Buenas Prácticas

- Crear PRs apenas se empieza a trabajar ayuda a que el resto del equipo tenga visibilidad de en qué está cada uno y hace que sea más fácil ayudar a resolver problemas.
- Conviene incorporar con frecuencia los cambios de la branch padre a la que uno está usando para trabajar.
- Recordar que los PRs son también un espacio para compartir conocimiento.
- Cuando se revisa el código de otra persona siempre hay que pensar en dar feedback constructivo y libre de ego.
- Es conveniente acompañar el manejo de PRs con herramientas de **integración continua** para asegurar la calidad del mismo.

Práctica de Resolución de Conflictos

WORKSHOP: Resolviendo Conflictos



WORKSHOP: Resolviendo Conflictos

```
$ git pull origin main
From github.com:plorenzatto/deleteme
 * branch            main       -> FETCH_HEAD
CONFLICT (add/add): Merge conflict in main.py
Auto-merging main.py
Automatic merge failed; fix conflicts and then commit the
result.

$ cat main.py
# main.py
if __name__ == '__main__':
<<<<<<< HEAD
    print("Hola Mundo!!!")
=======
    print("Hola Mundo")
>>>>>>> ff6c38b6717d483f0f57340589fb2c31037b0f15

$ [...] # Aca se arregla el archivo.

$ cat  main.py
# main.py
if __name__ == '__main__':
    print("Hola Mundo!!!")
```

```
$ [...] # Aca se arregla el archivo.

$ cat  main.py
# main.py
if __name__ == '__main__':
    print("Hola Mundo!!!")

$ git add main.py                    # Agregamos el archivo
$ git commit -am "Fix conflict"
[more-changes f198824] Fix conflict
$ git push                           # Pusheamos el cambio
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 397 bytes | 397.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local
object.
To github.com:plorenzatto/deleteme.git
   bae6936..f198824  more-changes -> more-changes
```

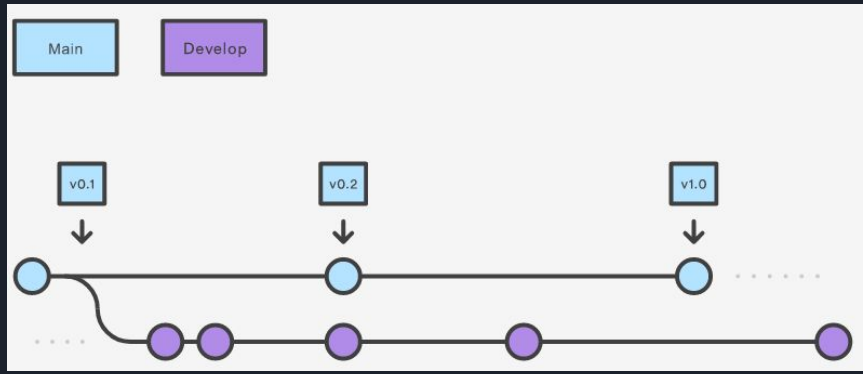
Gitflow

Gitflow: ¿Qué es?

- Gitflow es una forma de manejo de branches
- El branch `main` contiene los releases oficiales del código.
- develop sirve para integrar los cambios que se vayan agregando.

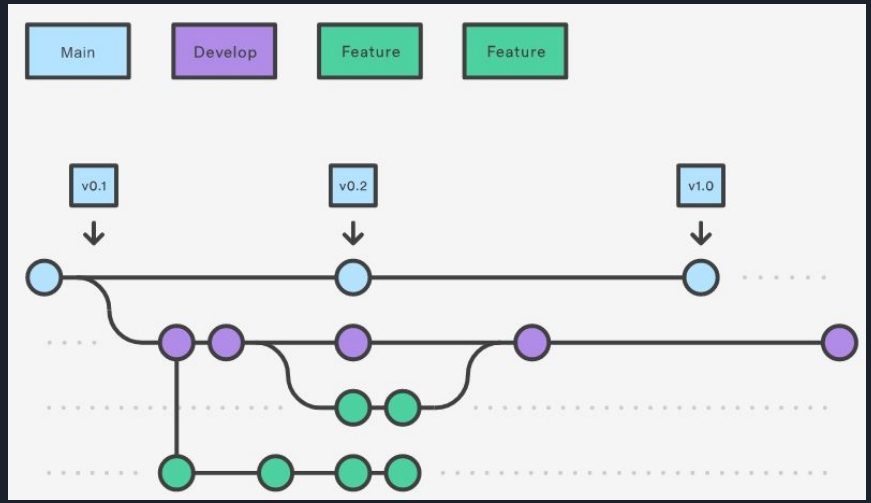
Gitflow: Main & Develop branches

- El punto de partida para Gitflow son los branches `main` (o `master`) y develop.
- `main` contiene los releases oficiales del código.
- `develop` sirve para integrar los cambios que se vayan agregando.



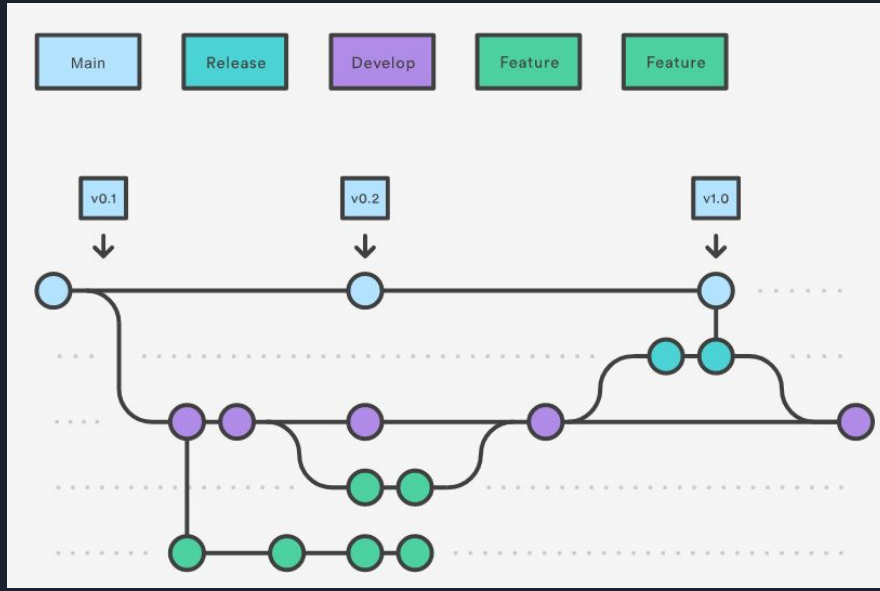
Git Flow: Feature branches

- Todas las features nuevas deben residir en su propia branch que salga de develop.
- Permiten que distintos desarrolladores puedan trabajar a la vez en distintos features.
- Una vez que el feature esté listo se mergea contra develop.



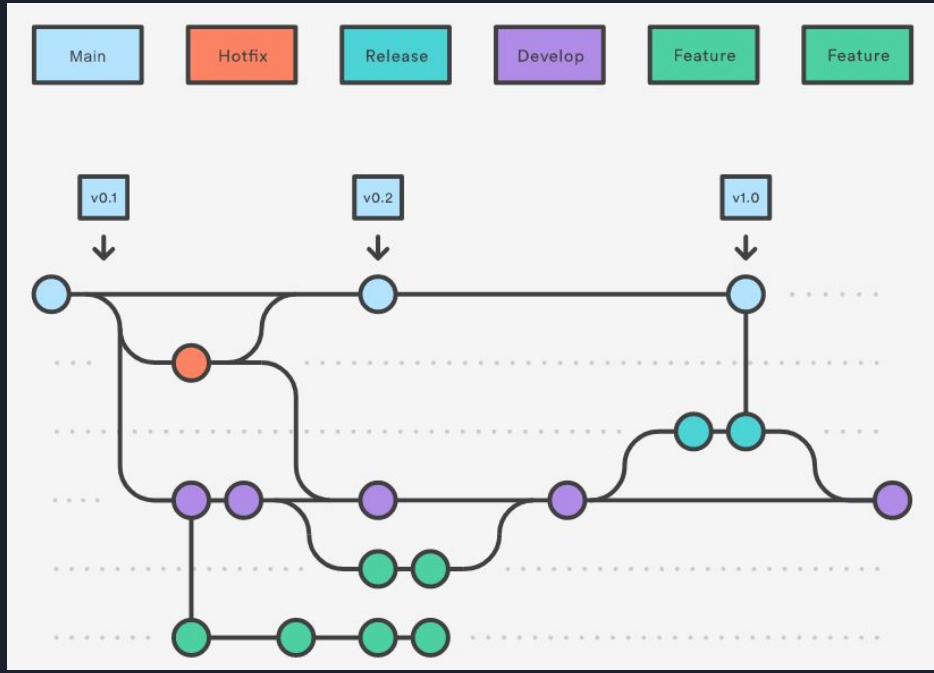
Git Flow: Release branches

- Estas tienen el objetivo de preparar releases.
- No reciben nuevos de features.
- Permiten que un equipo pueda seguir puliendo un release mientras el resto del equipo trabaja en features del próximo release.
- Luego de que se mergea el release branch los cambios realizados deben traerse a develop.



Git Flow: Hotfix branches

- Sirven para reparar rápidamente las releases en producción.
- Salen de la branch main en lugar de develop.
- Luego de que se mergea el hotfix branch los cambios realizados deben traerse a develop.



Fin
¿Preguntas?



Referencias y Tutoriales

- https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control
- https://www.edureka.co/blog/what-is-git/
- https://www.earthdatascience.org/courses/intro-to-earth-data-science/git-github/github-collaboration/how-to-submit-pull-requests-on-github/
- https://www.atlassian.com/es/git/tutorials/using-branches
- https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control
- https://www.atlassian.com/git/tutorials/using-branches/merge-conflicts
- https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow
- https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request
- https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews